15.4 Application-level caching

Application caches are a necessity for high-performance enterprise applications, and this section is going to explore this topic in greater detail. No matter how well tuned a database engine is, the statement response time is highly dependent on the incoming database load. A traffic spike incurs a high contention of database system resources, which can lead to higher response times.

For instance, most internet applications expose a sitemap which is used by Search Engine bots to index the content of the site in question and make it available for searches. From a business perspective, having a high page rank is highly desirable because it can translate to more revenue. However, the Search Engine bot can generate a sudden traffic spike, which, in turn, can lead to a spike in transaction response time. Unfortunately, high response times can affect the site page rank. That being said, the translation response time must be relatively low even during high traffic loads.

The application-level cache can, therefore, level up traffic spikes because the cache fetching complexity is O(1). More, if the application-level cache holds a significant portion of the entire data set, the application can still work (even if in a read-only mode) when the database is shut down for maintenance or due to a catastrophic event.

15.4.1 Entity aggregates

In a relation database, data is normalized, and, for a complex enterprise application, it is usually spread across multiple tables. On the other hand, the business logic might operate on entity graphs which assemble information from various database tables.

To better visualize the entity aggregate, consider the following diagram depicting all entities associated with a single `Post` in a particular forum.

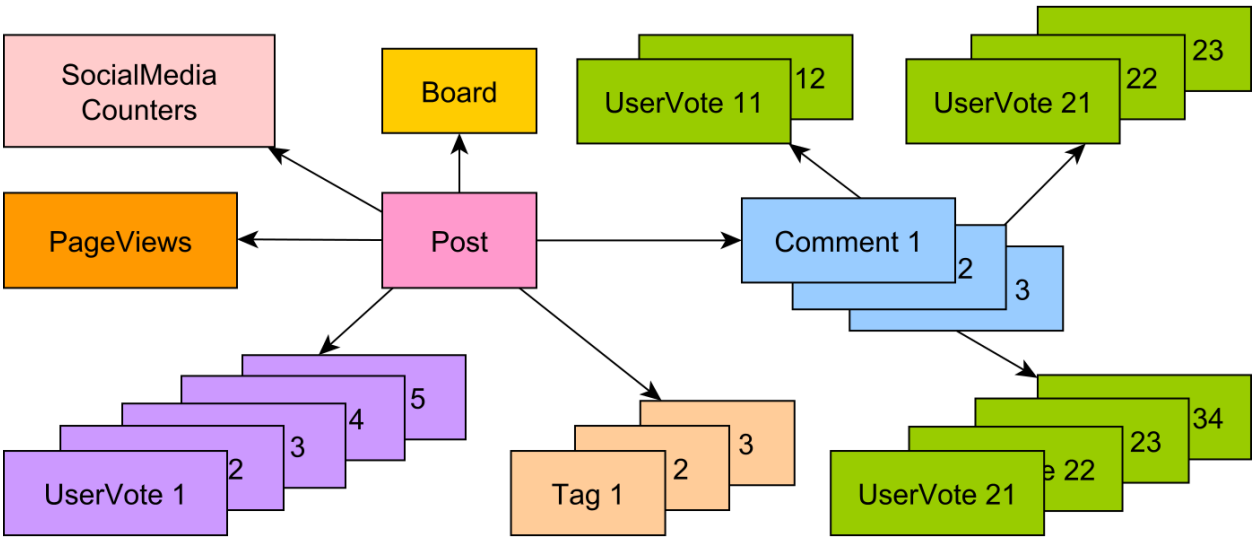

Figure 15.7: Entity aggregates

The `Post` entity is the root since all other entities relate to it, either directly or indirectly. A `Post` belongs to a `Board` entity, and it can have several `Tag(s)`. The `PageViews` entity summarizes statistics about how popular a given `Post` might be. There is also a `SocialMediaCounters` entity to hold the number of shares for social media platforms. Users can add `Comment(s)` to a `Post`, and they can also cast a vote on both the `Post` or the `Comment` entity.

The sitemap contains the list of all `Post(s)` so that Search Engines can index all questions and answers. When a `Post` is requested, the whole entity aggregate is required to render the display. Without application-level caching, the data access layer would have to either join all the associated entities or use secondary select statements.

To avoid a Cartesian Product, the `Post` entity should be joined to its `Tag(s)`, as well as with other `many-to-one` relationships (e.g. `Board`, `PageViews`, `SocialMediaCounters`). A secondary query is used to fetch the `UserVote(s)` associated with the current `Post`. The `Comment(s)` can be fetched with a secondary select, and this is desirable since there might be many `Comment(s)`, and the secondary query can better use pagination. The `Comment` query could also join the `UserVote(s)` so that these two entities are fetched with a single query as well.

15.4.2 Distributed key-value stores

While the underlying data resides in the relational database, the entity aggregate can also be saved in a distributed cache, such as Redis or Memcached. Key-value stores are optimized for storing data structures in memory, and the lookup complexity is O(1). This is ideal for high-performance enterprise applications since response time can stay low even during unforeseen traffic spikes.

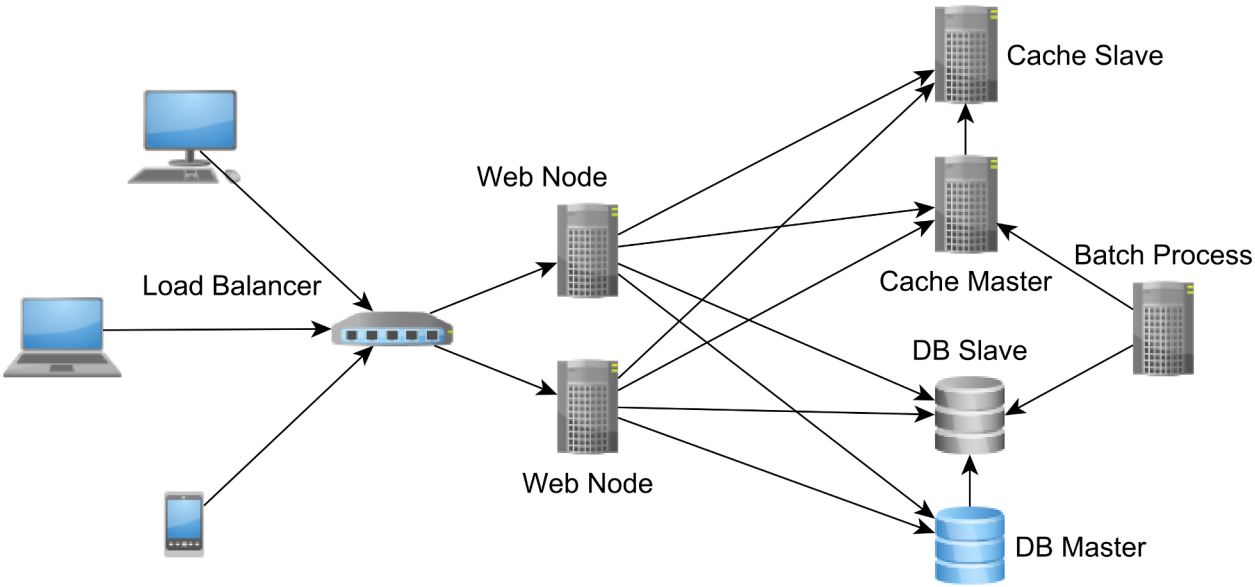

Figure 15.8: Application-level cache integration

The relational database is still the system of record, while the key-value caching solution is used as an alternate data provider.

15.4.3 Cache synchronization patterns

Unfortunately, duplicating data among two data sources is not without issues. Ideally, both the relational database and the application-level cache should be in sync, every update being applied synchronously to both data stores. In reality, not all business use cases have strict consistency requirements.

For instance, the `PageViews` and the `SocialMediaCounters` can be updated periodically by a batch processor which can aggregate the database counters and update the cache entry with the latest aggregated values. On the other hand, some actions need a more strict consistency guarantees. For `Comment` entries, read-your-writes consistency is needed because otherwise users might miss their own changes.

Caching is always about trade-offs, and not all business use cases are equal in terms of consistency guarantees. For strict consistency, some transactions might need to bypass the cache entirely and read from the database.

Once a user or a batch process makes a change to the underlying database records, the cache needs to be updated as well. As explained in the cache concurrency strategies section, there are several ways to implement the synchronization logic.

15.4.4 Synchronous updates

If cache-aside is being used, the business logic must update both the database and all associated cache entries in the same transaction. Because most key-value stores do not use XA transactions, the cache entries can be either invalidated (in which case there is no risk of reading stale data), or they can be updated after the database transaction has been committed (in which case there is a slight time interval when a concurrent transaction can read a stale entry from the cache).

For the previous Domain Model, `Comment` entities should be processed synchronously. Adding or modifying a `UserVote` entry can also be done synchronously, or at least for the comments that are associated with the currently logged user.

15.4.5 Asynchronous updates

If eventual consistency is tolerated, then asynchronous updates are also a viable solution, and the application logic can be simplified since the caching logic is decoupled from business logic. This is also necessary when there are multiple data stores that need to be updated according to the latest changes that happened in the database. For instance, the enterprise

application might need to propagate changes to a cache, an in-memory data processing framework (e.g. Spark) which might monitor the forum for spam messages, or to a data warehouse. In this case, the changes must be captured from the database and propagated to all other subsystems that are interested in being notified about these updates.

15.4.5.1 Change data capture

In database terminology, change data capture (CDC) is an assembly of patterns that are responsible for recording database changes.

One solution is to record the timestamp version of every row that is either inserted, updated, or deleted. This pattern only works if records are not actually physically removed, but instead, they are simply marked as deleted (soft deleting), and hidden away from any database query.

Another implementation would be to capture changes using database triggers so that an event is recorded whenever a row is inserted, updated, or deleted. Unfortunately, triggers might slow down write operations which is undesirable especially if there is only one database Master node because the longer the write transactions take, the less throughput the Master node will accommodate.

A more efficient approach is to use a framework that can parse the database transaction log. Unlike database triggers, this approach does not incur any additional performance penalty for write operations since the transaction log is being parsed asynchronously. The only drawback is that not all database support this natively, and the transaction log entries can change from one database version to the other.

Oracle

Oracle GoldenGate[a] is a change data capture tool that can be used either for database replication or as an ETL (extract, transform, and load) process in order to feed a data warehouse. Another approach is to use Databus[b], which is an open-source framework developed by Linkedin for log mining.

[a] http://www.oracle.com/us/products/middleware/data-integration/goldengate/overview/index.html
[b] https://github.com/linkedin/databus

SQL Server

Since version 2008, SQL Server offers a Change Data Capture[a] solution that can be configured at the database, table, or even column level.

[a] https://msdn.microsoft.com/en-us/library/cc627369.aspx

PostgreSQL

Even if there is no native CDC solution, PostgreSQL 9.4 has introduced logical decoding[a] which can be used for extracting row-level modifications.

[a] http://www.postgresql.org/docs/current/static/logicaldecoding.html

MySQL

There are multiple solutions that are able to parse the MySQL binary log, and the most notable is Databus which supports both Oracle and MySQL.

Denormalization ripple effect

In the previous Domain Model, storing the `Board` and the list of `Tag(s)` associated with every particular `Post` entity graph is appropriate only if the `Board` and the `Tag` are practically immutable. Otherwise, changing the `Board` entity could ripple throughout the cache, causing a large number of entries to be updated as well. This problem is even more acute if cache entry invalidation is being used. For `Tag(s)`, the `Post` aggregate should store only a list of `Tag` identifiers, the actual `Tag` names being resolved upon fetching the `Post` entity aggregate from the cache.

On the other hand, `Comment(s)` and `UserVote(s)` are more related to a single `Post` entry, so they are more suitable for being stored in the `Post` entity aggregate. To avoid the ripple effect, the `UserVote` entity should only contain virtually immutable user-related columns (e.g. user identifier).

The higher the data denormalization degree associated with entity aggregates, the bigger the data change ripple. Therefore, it is good practice to avoid storing associations that might be shared among many entity graphs cache entries.